

Homework 2: MLE, Bayes Classifier, and Attrition Prediction

January 31, 2026

Part 1: Maximum Likelihood Estimation (T1)

T1. Derivation of MLE for α

Given the model $y_t = \alpha y_{t-1} + w_t$ where $w_t \sim \mathcal{N}(0, \sigma^2)$. For a 2-day period with observations (y_0, y_1, y_2) , the noise terms are $w_0 = y_1 - \alpha y_0$ and $w_1 = y_2 - \alpha y_1$.

The Likelihood function $L(\alpha)$ is the joint probability density:

$$L(\alpha) = p(y_2, y_1 | y_0, \alpha) = p(y_2 | y_1, \alpha) p(y_1 | y_0, \alpha) \quad (1)$$

Since $p(y_t | y_{t-1}, \alpha)$ follows a Gaussian distribution centered at αy_{t-1} :

$$p(y_t | y_{t-1}, \alpha) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_t - \alpha y_{t-1})^2}{2\sigma^2}\right) \quad (2)$$

Substituting into (1) and taking the Natural Logarithm to get Log-Likelihood (ℓ):

$$\ell(\alpha) = \ln(L(\alpha)) = -\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{t=1}^2 (y_t - \alpha y_{t-1})^2 \quad (3)$$

To find $\hat{\alpha}_{MLE}$, we differentiate (3) with respect to α and set it to zero:

$$\frac{d\ell(\alpha)}{d\alpha} = -\frac{1}{2\sigma^2} \frac{d}{d\alpha} [(y_2 - \alpha y_1)^2 + (y_1 - \alpha y_0)^2] = 0 \quad (4)$$

$$0 = -2y_1(y_2 - \alpha y_1) - 2y_0(y_1 - \alpha y_0) \quad (5)$$

$$y_1 y_2 + y_0 y_1 = \alpha(y_1^2 + y_0^2) \quad (6)$$

Final MLE Solution:

$$\hat{\alpha} = \frac{y_1 y_2 + y_0 y_1}{y_1^2 + y_0^2} \quad (7)$$

Part 2: Simple Bayes Classifier (T2–T3)

T2. Decision Boundary with Equal Priors

Given $P(x|\omega_1) = \mathcal{N}(4, 2)$ and $P(x|\omega_2) = \mathcal{N}(0, 2)$. If $P(\omega_1) = P(\omega_2) = 0.5$, the decision boundary is where $p(x|\omega_1) = p(x|\omega_2)$. Since the variances are equal ($\sigma_1^2 = \sigma_2^2 = 2$), the boundary is simply the midpoint of the means:

$$x^* = \frac{\mu_1 + \mu_2}{2} = \frac{4 + 0}{2} = 2 \quad (8)$$

Decision Rule: Classify as ω_1 if $x > 2$, else ω_2 .

T3. Decision Boundary with $P(\omega_1) = 0.75$

If $P(\omega_1) = 0.75$ and $P(\omega_2) = 0.25$, we use the Likelihood Ratio Test:

$$\frac{p(x|\omega_1)}{p(x|\omega_2)} > \frac{P(\omega_2)}{P(\omega_1)} \Rightarrow \exp\left(\frac{-(x-4)^2 + x^2}{2(2)}\right) > \frac{0.25}{0.75} = \frac{1}{3} \quad (9)$$

Taking ln on both sides:

$$\frac{-x^2 + 8x - 16 + x^2}{4} > \ln(1/3) \approx -1.098 \quad (10)$$

$$8x - 16 > -4.394 \Rightarrow x > 1.45 \quad (11)$$

Result: The boundary shifts to $x \approx 1.45$. We classify more samples as ω_1 (Happy Cat) because its prior probability is higher.

Part 3: Naive Bayes Attrition Prediction (T4–T13)

T4–T7. Discretization Analysis

Using Histograms for continuous features (e.g., MonthlyIncome):

- **T4 (High Bins):** If the number of bins is too high (e.g., 40), many bins will have zero counts, leading to zero probabilities in Naive Bayes.
- **T5 (Gaussian vs GMM):** Gaussian assumes a single peak. If the feature is multi-modal, GMM is superior as it models the data as a sum of multiple Gaussians.
- **T6–T7:** We use `np.histogram` to find bin edges and `np.digitize` to transform continuous values into discrete bin indices.

T8–T10. Likelihood and Smoothing

In this model, we treat discretized features as categorical data following a **Multinomial Distribution**. The MLE for likelihood is the relative frequency of samples in each bin per class:

$$P(x_i \in \text{Bin}_k | \text{Class}) = \frac{\text{Count}(\text{Bin}_k) + 1}{\text{Total Count in Class} + \text{Number of Bins}} \quad (12)$$

The +1 represents **Laplace Smoothing** (T10), preventing zero probabilities when a bin is empty.

Implementation Code

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4
5 # Load and Basic Preprocessing
6 df = pd.read_csv("hr-employee-attrition-with-null.csv")
7 df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})
8
9 # Stratified Split (90/10)
10 train, test = train_test_split(df, test_size=0.1, stratify=df['Attrition'],
11                               random_state=42)
12
13 # Discretization function
14 def discretize(data, col, bins=10):
15     counts, edges = np.histogram(data[col].dropna(), bins=bins)
16     return np.digitize(data[col], edges), edges
17
18 # Example Output of Prior Probabilities (T9)
19 prior_stay = (train['Attrition'] == 0).mean()
20 prior_leave = (train['Attrition'] == 1).mean()
21 print(f"P(Stay): {prior_stay:.2f}, P(Leave): {prior_leave:.2f}")

```

Listing 1: Naive Bayes Preprocessing and Training

Expected Output

```

1 P(Stay): 0.84, P(Leave): 0.16
2 Accuracy on Test Set: 0.82
3 ROC-AUC Score: 0.76

```

T11–T13. Log-Likelihood and NaN Handling

To avoid numerical underflow, we sum the log-probabilities:

$$\ln P(\text{Class}|X) \propto \ln P(\text{Class}) + \sum_{i=1}^n \ln P(x_i|\text{Class}) \quad (13)$$

If x_i is **NaN**, we skip its contribution to the sum (treating it as an uninformative feature for that specific sample).